

Transformers in Machine Learning

Architecture, Core Concepts, Self-Attention, and Applications

Source Material: GeeksforGeeks — Full Comprehensive Documentation

1. Overview and Definition

A **Transformer** is a neural network architecture used for various machine learning tasks, especially in **Natural Language Processing (NLP)** and **Computer Vision (CV)**. It focuses on understanding complex relationships within data to process information more effectively.

Key Characteristics:

- Uses **attention mechanisms** to capture intricate relationships between inputs regardless of distance.
- Processes **entire sequences at once in parallel** instead of step-by-step sequential processing.
- Significantly improves performance on tasks involving long-range context and complex dependencies.
- Widely deployed across NLP, computer vision, speech recognition, and generative AI.

2. The Need for Transformer Models

Traditional sequence-to-sequence models (such as Recurrent Neural Networks — RNNs and Long Short-Term Memory networks — LSTMs) process data sequentially word-by-word. This introduces two fundamental bottlenecks:

1. **Sequential Bottleneck & Vanishing Gradients in RNNs:** RNNs analyze words one at a time. As sequence length increases, RNNs suffer from the **vanishing gradient problem**, leading to long-term memory loss.
 - *Example:* In the sentence: “XYZ went to France in 2019 when there were no cases of COVID and there he met the president of that country”, the phrase “that country” refers back to “France”. RNNs struggle to link such distant words over long sequence gaps.
2. **Limitations of LSTMs:** While LSTMs introduced memory cells and gating mechanisms to mitigate vanishing gradients, they **still process words sequentially**. They cannot process an entire sentence simultaneously, preventing hardware parallelization (GPU acceleration).
3. **Information Bottleneck in Traditional Seq2Seq Models:** Traditional Seq2Seq architectures compress the entire input sequence into a single, fixed-size context vector. For long sentences, this creates a severe **information bottleneck**, causing crucial context to be lost during compression.
 - *Contextual Ambiguity Example:* The word “point” has completely different meanings in different contexts:
 - “The needle has a sharp point.” (→ Tip)
 - “It is not polite to point at people.” (→ Gesture)

Transformers solve this bottleneck by using **attention mechanisms** to directly access all tokens across the sequence without compression.

3. Core Architectural Concepts of Transformers

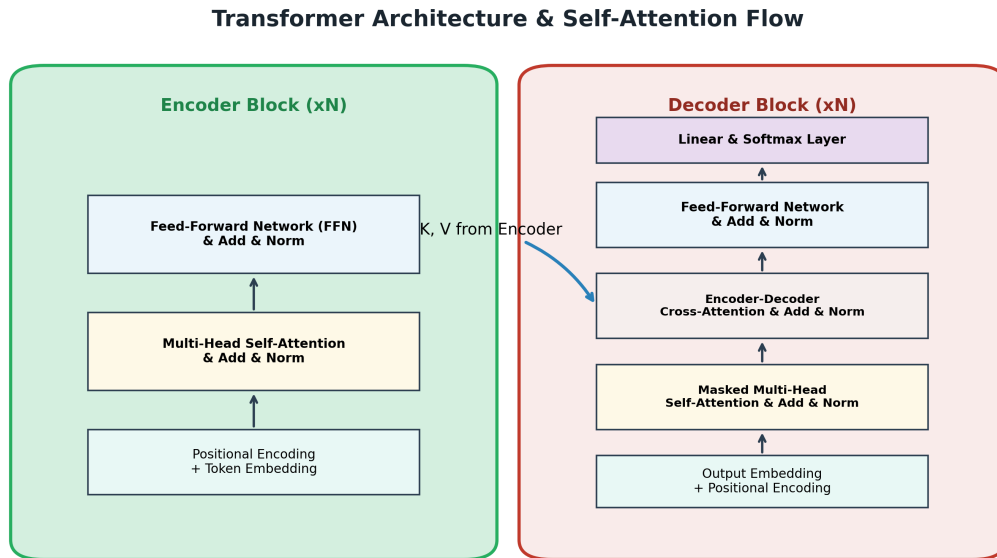


Figure 1: High-level Overview of the Transformer Encoder-Decoder Architecture and Self-Attention Flow.

3.1. Self-Attention Mechanism

The **self-attention mechanism** allows transformers to evaluate which words in a sentence are most relevant to each other. It computes a weighted representation of each token based on all other tokens using a **Scaled Dot-Product Attention**:

Each input token is projected into three trainable vectors:

- **Query (Q)**: Represents what the current token is looking for.
- **Key (K)**: Represents what information the token contains.
- **Value (V)**: Represents the actual content/features of the token.

The attention scores are calculated using the mathematical equation:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (1)$$

where d_k is the dimensionality of the key vectors. Scaling by $\frac{1}{\sqrt{d_k}}$ prevents dot products from growing excessively large in high dimensions, which would push the Softmax function into regions with extremely small gradients.

3.2. Multi-Head Attention

Instead of computing a single attention function, Transformers employ **Multi-Head Attention** running in parallel. The Query, Key, and Value vectors are projected into multiple lower-dimensional subspaces (heads). Each head focuses on different types of relationships (e.g., syntactic vs. semantic, short-range vs. long-range), producing a rich, multi-faceted representation.

3.3. Positional Encoding

Because Transformers process all tokens in parallel simultaneously, they lack an inherent sense of word order or sequence position. To provide positional context, **Positional Encodings** are added directly to the input token embeddings. These encodings use trigonometric functions (sine and cosine of different frequencies) or learned embeddings to uniquely identify token positions within a sequence.

3.4. Position-wise Feed-Forward Networks (FFN)

Each block in the Encoder and Decoder contains a Position-wise Feed-Forward Network consisting of two linear transformations with a **ReLU activation** in between, applied independently to each token position:

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2 \quad (2)$$

This position-wise transformation further refines the feature representations learned during the attention steps.

3.5. Add & Norm (Residual Connections and Layer Normalization)

Every sub-layer (Self-Attention and FFN) is wrapped with a residual connection followed by Layer Normalization:

- **Residual Connection (Add):** Adds the original input of the sub-layer directly to its output: $\text{Output} = x + \text{SubLayer}(x)$. This preserves signal integrity, enables direct gradient flow during backpropagation, and prevents vanishing gradients in deep networks.
- **Layer Normalization (Norm):** Normalizes feature activations across channels for each sample to stabilize training and accelerate convergence.

3.6. Embeddings and Weight Sharing

Transformers convert raw text tokens into continuous numerical vectors called **embeddings**:

- Encoder input tokens and Decoder input tokens are mapped into trainable embedding vectors.
- The model shares the **same weight matrix** across Encoder embeddings, Decoder embeddings, and the final linear projection layer before Softmax.
- Embedding values are scaled by $\sqrt{d_{\text{model}}}$ prior to adding positional encodings to maintain numerical stability.

3.7. Encoder-Decoder Architecture and Attention Types

The Transformer follows an Encoder-Decoder structure (e.g., translating French “*Je suis étudiant*” to English “*I am a student*”):

1. **Encoder Self-Attention:** Q, K, V all originate from the previous layer of the Encoder. Every token can attend to every other token in the input sequence, capturing full bi-directional context.
2. **Decoder Self-Attention (Masked):** Q, K, V come from the Decoder. Future tokens are masked out (set to $-\infty$ before Softmax) so each position can only attend to earlier positions. This preserves auto-regressive generation.
3. **Encoder-Decoder Cross-Attention:** Queries (Q) come from the Decoder, while Keys (K) and Values (V) originate from the final output of the Encoder. This enables the Decoder to focus on relevant sections of the source sequence while generating output.

3.8. Softmax Layer for Output Prediction

To predict the next token, the Decoder output passes through a linear projection layer (sharing weights with token embeddings) to compute logits, followed by a **Softmax function**:

$$P(y_t = k \mid y_{<t}, X) = \frac{\exp(z_k)}{\sum_{j=1}^{V_{\text{vocab}}} \exp(z_j)} \quad (3)$$

The token with the highest probability is selected as the predicted next word.

4. Applications of Transformers

- **NLP Tasks:** Machine translation, text summarization, Named Entity Recognition (NER), sentiment analysis, and question answering (LLMs like GPT, BERT, T5).
- **Speech Recognition:** Processing raw audio signals to convert spoken language into transcribed text.
- **Computer Vision:** Vision Transformers (ViT) for image classification, object detection (DETR), and generative image models.

- **Recommendation Systems:** Capturing long-term user behavior sequences to provide personalized recommendations.
- **Text and Music Generation:** Generating articles, code generation, and composing musical arrangements.

5. Limitations of Transformers

- **High Computational Power:** Training requires substantial GPU resources and energy, making training expensive.
- **Memory Consumption:** Quadratic memory complexity $O(N^2)$ with respect to sequence length N due to the full attention matrix.
- **Large Dataset Requirement:** Lacks inductive bias (like CNN spatial locality or RNN temporal order) and needs massive pre-training datasets to perform well.
- **Input Length Limits:** Standard models have fixed context windows (e.g., 512 or 2048 tokens), making extremely long documents difficult to process without special modifications.
- **Black-Box Nature:** High parameter counts reduce model interpretability.
- **Overfitting Risk:** Prone to overfitting on small datasets if not properly regularized with dropout and weight decay.